

第二次作业：贪心与树搜索

姓名：郑文翔

学号：25S003050

1 找零问题：25、10、5、1 美分

1.1 贪心策略

每一步选择当前不超过剩余金额的最大面额硬币。也就是依次尽可能多地使用 25 美分、10 美分、5 美分和 1 美分硬币。

1.2 算法描述

设需要找零金额为 n ，则：

$$q_{25} = \left\lfloor \frac{n}{25} \right\rfloor, \quad n = n \bmod 25,$$

$$q_{10} = \left\lfloor \frac{n}{10} \right\rfloor, \quad n = n \bmod 10,$$

$$q_5 = \left\lfloor \frac{n}{5} \right\rfloor, \quad n = n \bmod 5,$$

$$q_1 = n.$$

硬币总数为

$$q_{25} + q_{10} + q_5 + q_1.$$

1.3 伪代码

算法 1 美元硬币系统的贪心找零算法

输入： 找零金额 n

输出： 四种面额硬币数量 $(q_{25}, q_{10}, q_5, q_1)$

1: $q_{25} \leftarrow \lfloor n/25 \rfloor$

2: $n \leftarrow n \bmod 25$

3: $q_{10} \leftarrow \lfloor n/10 \rfloor$

4: $n \leftarrow n \bmod 10$

5: $q_5 \leftarrow \lfloor n/5 \rfloor$

6: $n \leftarrow n \bmod 5$

7: $q_1 \leftarrow n$

8: **return** $(q_{25}, q_{10}, q_5, q_1)$

1.4 最优性证明

对于小于 25 美分的部分， $10 = 5 + 5$ ，用一个 10 美分硬币优于用两个 5 美分硬币； $5 = 1 + 1 + 1 + 1 + 1$ ，用一个 5 美分硬币优于用五个 1 美分硬币。因此当剩余金额达到相应面额时，选择最大面额是最优的。

当 $n \geq 25$ 时，如果不用 25 美分硬币而凑出 25 美分，最少需要

$$10 + 10 + 5$$

共 3 枚硬币；而一个 25 美分硬币只需要 1 枚。故最优解中可以包含 25 美分硬币。

选择一枚硬币后，剩余金额仍然是同类找零问题。如果剩余部分不是最优解，则可用更优剩余方案替换，从而得到更优的整体方案，矛盾。因此问题具有最优子结构。

综上，面额为 25, 10, 5, 1 时，贪心算法能够得到最少硬币数。

2 找零问题：面额为 $c^0, c^1, \dots, c^k$

2.1 贪心策略

每一步选择当前不超过剩余金额的最大面额 c^i 。

2.2 伪代码

算法 2 幂次面额系统的贪心找零算法

输入： 找零金额 n ，底数 c ，最大指数 k

输出： 各面额硬币数量数组 q 及硬币总数 $count$

```

1:  $count \leftarrow 0$ 
2: for  $i \leftarrow k$  downto 0 do
3:    $q_i \leftarrow \lfloor n/c^i \rfloor$ 
4:    $n \leftarrow n \bmod c^i$ 
5:    $count \leftarrow count + q_i$ 
6: end for
7: return  $(q, count)$ 

```

2.3 最优性证明

由于

$$c^i = c \cdot c^{i-1},$$

一个高一级面额硬币等于 c 个低一级面额硬币。若 $c > 1$ ，使用一个高面额硬币比使用 c 个低面额硬币需要更少硬币；若 $c = 1$ ，所有面额相同，任意选择都不影响最优性。

在任意最优解中，同一种面额 c^i 的硬币数量不应达到 c 枚。若有 c 枚，则可替换为一枚 c^{i+1} ，硬币数量减少。因此最优解与 c 进制表示一致。贪心算法从高位到低位选择硬币，正对应于将金额写成 c 进制形式，所以得到最优解。

3 构造贪心失败的硬币面额

取硬币面额为

$$1, 3, 4.$$

该面额集合包含 1 美分，因此任意金额都可以找零。

设找零金额为 6。贪心算法先选择 4，剩余 2，只能使用两个 1：

$$6 = 4 + 1 + 1,$$

共 3 枚硬币。

但最优解为

$$6 = 3 + 3,$$

共 2 枚硬币。因此对于面额集合 $\{1, 3, 4\}$ ，贪心算法不能保证得到最优解。

4 树上结点染色最小代价

4.1 问题分析

树有 n 个结点，根为 r 。每个结点 i 权值为 w_i ，染色时间为 t_i ，代价为

$$w_i t_i.$$

每个结点只有在父结点已经染色后才能染色。目标是最小化总代价。

4.2 贪心策略

将若干已经确定相对顺序的结点看作一个结点块 B 。定义

$$W(B) = \text{块中结点权值和}, \quad S(B) = \text{块中结点个数}.$$

每次选择平均权值

$$\frac{W(B)}{S(B)}$$

最大的非根结点块，将其合并到父结点块之后。

4.3 贪心依据

考虑两个结点块 A, B 。若先 A 后 B ，则 B 被整体推迟 $S(A)$ 个单位，额外代价为

$$S(A)W(B).$$

若先 B 后 A ，额外代价为

$$S(B)W(A).$$

先 A 后 B 更优当且仅当

$$S(A)W(B) \leq S(B)W(A),$$

即

$$\frac{W(A)}{S(A)} \geq \frac{W(B)}{S(B)}.$$

所以平均权值大的结点块应排在前面。

4.4 贪心选择性与最优子结构

若某一顺序中平均权值更大的块 B 排在较小的块 A 后面，则交换 A, B 可降低或不增加总代价。因此存在一个最优方案优先安排平均权值最大的块，满足贪心选择性。

合并一个结点块到其父块后，剩余问题仍是同类树上染色问题。如果剩余部分不是最优，则可替换为更优方案，得到整体更优方案，矛盾。因此具有最优子结构。

4.5 伪代码

算法 3 树上结点染色的平均权值合并算法

输入： 根为 r 的树 T ，每个结点权值 w_i

输出： 最小染色总代价 ans

```

1: for 每个结点  $i \in T$  do
2:    $W_i \leftarrow w_i, S_i \leftarrow 1$ 
3:    $active_i \leftarrow true$ 
4:    $parentBlock_i \leftarrow parent_i$ 
5: end for
6:  $ans \leftarrow 0$ 
7: for  $t \leftarrow 1$  to  $n - 1$  do
8:   在所有活跃非根块中选取  $x$ ，使  $W_x/S_x$  最大
9:    $p \leftarrow parentBlock_x$ 
10:   $ans \leftarrow ans + S_p \cdot W_x$ 
11:   $W_p \leftarrow W_p + W_x, S_p \leftarrow S_p + S_x$ 
12:   $active_x \leftarrow false$ 
13:  for 每个满足  $parentBlock_y = x$  的活跃块  $y$  do
14:     $parentBlock_y \leftarrow p$ 
15:  end for
16: end for
17: return  $ans$ 

```

若每次线性扫描寻找最大平均权值块，时间复杂度为 $O(n^2)$ ，空间复杂度为 $O(n)$ 。

5 冒险者完成村庄委托任务

5.1 贪心策略

每次选择当前尚未完成任务中难度最大的任务，使用能力值不小于该任务难度且能力值最小的未使用冒险者完成它。选中该冒险者后，让他在每个村庄中至多完成一个难度不超过其能力值的任务，并优先完成该村庄中较难的可完成任务。

5.2 证明

设当前最大难度任务为 d 。任意可行方案都必须使用某个能力值至少为 d 的冒险者。若贪心选择的冒险者能力值为 p_a ，且 p_a 是所有可行冒险者中最小的，则任何使用能力值更高冒险者完成该任务的方案，都可用该冒险者替换而不增加代价。因此该选择安全。

冒险者一旦出击，代价固定为其能力值，完成更多任务不会增加代价，因此应尽量让其完成更多可完成任务。完成这一轮任务后，剩余任务与剩余冒险者构成同类子问题，故具有最优子结构。

5.3 伪代码

算法 4 冒险者委托任务的贪心调度算法

输入： 各村庄任务集合 $A_1, A_2, \dots, A_m$ ，冒险者能力数组 P

输出： 完成所有任务的最小总代价；若不可行则报告无解

```

1: 将每个  $A_j$  按任务难度降序排序
2: 将  $P$  按能力值升序排序，并标记所有冒险者未使用
3:  $ans \leftarrow 0$ 
4: while 仍存在未完成的任务 do
5:   取当前所有未完成的任务中的最大难度  $d$ 
6:   选择未使用且满足  $P_i \geq d$  的最小能力冒险者  $i$ 
7:   if 不存在这样的冒险者 then
8:     return 无可行方案
9:   end if
10:  标记冒险者  $i$  已使用， $ans \leftarrow ans + P_i$ 
11:  for  $j \leftarrow 1$  to  $m$  do
12:    if  $A_j$  中存在难度不超过  $P_i$  的未完成的任务 then
13:      完成其中难度最大的一个任务
14:    end if
15:  end for
16: end while
17: return  $ans$ 

```

设任务总数为 K ，朴素实现时间复杂度可记为 $O(nK + n^2)$ ，空间复杂度为 $O(K + n)$ 。

6 海岸线雷达覆盖小岛

6.1 问题转化

雷达安装在 x 轴上，位置为 $(a, 0)$ 。小岛 i 的位置为 (x_i, y_i) 。雷达覆盖小岛需满足

$$(a - x_i)^2 + y_i^2 \leq d^2.$$

若 $y_i > d$ ，则无解。否则可得雷达位置区间

$$[l_i, r_i] = \left[x_i - \sqrt{d^2 - y_i^2}, x_i + \sqrt{d^2 - y_i^2} \right].$$

原问题转化为：在数轴上选择最少的点，使每个区间至少包含一个点。

6.2 贪心策略与证明

将区间按右端点从小到大排序。每次对当前未覆盖且右端点最小的区间，在其右端点放置雷达。

设当前最早结束的未覆盖区间为 $[l_1, r_1]$ 。任何可行方案都必须在该区间内放置一个点。将覆盖该区间的点移动到 r_1 不会影响当前区间，并且由于 r_1 尽可能靠右，更有利于覆盖后续区间。因此该选择安全。放置一个雷达后，未覆盖区间构成同类子问题，具有最优子结构。

6.3 伪代码

算法 5 海岸线雷达覆盖算法

输入： 小岛坐标集合 I ，雷达半径 d

输出： 覆盖所有小岛所需的最少雷达数；若不可行则报告无解

```

1:  $Intervals \leftarrow \emptyset$ 
2: for 每个小岛  $(x_i, y_i) \in I$  do
3:   if  $y_i > d$  then
4:     return 无解
5:   end if
6:    $\Delta_i \leftarrow \sqrt{d^2 - y_i^2}$ 
7:   将区间  $(x_i - \Delta_i, x_i + \Delta_i)$  加入  $Intervals$ 
8: end for
9: 按右端点从小到大排序  $Intervals$ 
10:  $ans \leftarrow 0, last \leftarrow -\infty$ 
11: for 每个区间  $(l, r) \in Intervals$  do
12:   if  $last < l$  then
13:      $last \leftarrow r$ 
14:      $ans \leftarrow ans + 1$ 
15:   end if
16: end for
17: return  $ans$ 

```

时间复杂度为 $O(n \log n)$ ，空间复杂度为 $O(n)$ 。

7 哈密顿环问题

7.1 图的边集

根据图可记顶点为

$$V = \{1, 2, 3, 4, 5, 6, 7\}.$$

边集为

$$(1, 2), (1, 6), (1, 7), (2, 6), (6, 5), (5, 3), (3, 4), (4, 5), (4, 7).$$

7.2 深度优先搜索

以 1 为起点，按照编号从小到大扩展：

$$1 \rightarrow 2 \rightarrow 6 \rightarrow 5 \rightarrow 3 \rightarrow 4 \rightarrow 7.$$

此时已经经过所有顶点，且存在边 $(7, 1)$ ，因此得到哈密顿环

$$1 \rightarrow 2 \rightarrow 6 \rightarrow 5 \rightarrow 3 \rightarrow 4 \rightarrow 7 \rightarrow 1.$$

7.3 广度优先搜索

BFS 按路径长度逐层扩展。主要层次为：

$$(1),$$

$$(1, 2), (1, 6), (1, 7),$$

$$(1, 2, 6), (1, 6, 2), (1, 6, 5), (1, 7, 4),$$

继续扩展可得到完整路径

$$(1, 2, 6, 5, 3, 4, 7).$$

由于 7 与 1 相邻，故存在哈密顿环。结论同 DFS。

8 8-魔方：爬山法与最佳优先搜索

8.1 初始状态与目标状态

空格记为 0。初始状态为

$$S_0 = \begin{bmatrix} 2 & 3 & 0 \\ 1 & 8 & 5 \\ 7 & 4 & 6 \end{bmatrix},$$

目标状态为

$$G = \begin{bmatrix} 1 & 2 & 3 \\ 8 & 0 & 4 \\ 7 & 6 & 5 \end{bmatrix}.$$

采用启发函数

$$h(n) = \text{所有数字到目标位置的曼哈顿距离之和},$$

空格不计入距离。

8.2 搜索过程

一条从初始状态到目标状态的路径为:

$$\begin{bmatrix} 2 & 3 & 0 \\ 1 & 8 & 5 \\ 7 & 4 & 6 \end{bmatrix} \rightarrow \begin{bmatrix} 2 & 0 & 3 \\ 1 & 8 & 5 \\ 7 & 4 & 6 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 2 & 3 \\ 1 & 8 & 5 \\ 7 & 4 & 6 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 2 & 3 \\ 0 & 8 & 5 \\ 7 & 4 & 6 \end{bmatrix} \\ \rightarrow \begin{bmatrix} 1 & 2 & 3 \\ 8 & 0 & 5 \\ 7 & 4 & 6 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 2 & 3 \\ 8 & 4 & 5 \\ 7 & 0 & 6 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 2 & 3 \\ 8 & 4 & 5 \\ 7 & 6 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 2 & 3 \\ 8 & 4 & 0 \\ 7 & 6 & 5 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 2 & 3 \\ 8 & 0 & 4 \\ 7 & 6 & 5 \end{bmatrix}.$$

对应空格移动方向为

$$\boxed{\rightarrow \rightarrow \rightarrow \rightarrow \rightarrow \rightarrow \rightarrow}.$$

对应 $h(n)$ 值依次为

$$8, 7, 6, 5, 4, 3, 2, 1, 0.$$

因此, 爬山法和最佳优先搜索在该路径上都可以找到目标状态。

9 8-魔方: A* 算法

9.1 评价函数

取

$$g(n) = \text{从初始状态到当前状态的移动步数},$$

$$h(n) = \text{所有数字到目标位置的曼哈顿距离之和},$$

$$f(n) = g(n) + h(n).$$

9.2 主要步骤

A* 每次从 OPEN 表中取 $f(n)$ 最小的状态扩展。对本题, 上述路径中各步数值如下:

步数	$g(n)$	$h(n)$	$f(n)$
0	0	8	8
1	1	7	8
2	2	6	8
3	3	5	8
4	4	4	8
5	5	3	8
6	6	2	8
7	7	1	8
8	8	0	8

当 $h(n) = 0$ 时到达目标状态。故初始状态可经过 8 步到达目标状态。

10 迷宫问题: BFS 与 DFS

10.1 迷宫表示

迷宫为

E	E	T	W
E	W	T	E
E	E	E	T
W	T	E	E

起点为左上角 $(1, 1)$, 出口为右下角 $(4, 4)$ 。能量消耗为

$$E = 1, \quad W = 2, \quad T = 3.$$

10.2 BFS

规定搜索方向为

$\rightarrow \rightarrow \rightarrow .$

BFS 按层扩展, 找到的一条路径为

$$(1, 1) \rightarrow (1, 2) \rightarrow (1, 3) \rightarrow (1, 4) \rightarrow (2, 4) \rightarrow (3, 4) \rightarrow (4, 4).$$

对应格子类型为

$$E \rightarrow E \rightarrow T \rightarrow W \rightarrow E \rightarrow T \rightarrow E.$$

总能量为

$$1 + 1 + 3 + 2 + 1 + 3 + 1 = 12.$$

若不计起点能量，则为 11。

10.3 DFS

在同样的搜索方向顺序下，DFS 沿右侧路径深入，也可得到

$$(1, 1) \rightarrow (1, 2) \rightarrow (1, 3) \rightarrow (1, 4) \rightarrow (2, 4) \rightarrow (3, 4) \rightarrow (4, 4),$$

总能量同样为 12，若不计起点则为 11。

普通 BFS 和 DFS 找到的是可行路径，不保证能量最小。若要求最小能量路径，应使用统一代价搜索或 Dijkstra 算法。

11 最大完全子图

11.1 邻接关系

顶点集合为

$$V = \{a, b, c, d, e, g, h, i\}.$$

主要邻接关系为：

$$\begin{aligned} a &: \{b, c\}, & b &: \{a, g\}, & c &: \{a, d, g\}, \\ d &: \{c, g, e, h\}, & e &: \{d, g, h, i\}, \\ g &: \{b, c, d, e, h\}, & h &: \{d, e, g, i\}, & i &: \{e, h\}. \end{aligned}$$

11.2 搜索过程

从各顶点出发搜索：

- 以 a 开始，只能形成 $\{a, b\}$ 或 $\{a, c\}$ ，大小为 2；
- 以 b 开始，可形成 $\{b, g\}$ ，大小为 2；
- 以 c 开始，可形成 $\{c, d, g\}$ ，大小为 3；
- 以 d 开始，可形成 $\{d, e, g, h\}$ ，大小为 4；
- 以 e 开始，可形成 $\{e, h, i\}$ 或 $\{e, g, h\}$ ，大小为 3。

检查是否存在五元团：若要在 $\{d, e, g, h\}$ 基础上加入第五个顶点，则该顶点必须同时与 d, e, g, h 相邻。顶点 a, b, c, i 均不满足，因此不存在大小为 5 的团。

最大完全子图为

$$\boxed{\{d, e, g, h\}},$$

最大团大小为

$$\boxed{4}.$$

12 A* 搜索求 S 到 T 的最短路径

12.1 评价函数

取

$g(n)$ = 从 S 到 n 的实际路径代价,

$$h(n) = 0,$$

$$f(n) = g(n) + h(n) = g(n).$$

该启发函数不高估真实距离，满足 A* 要求。此时 A* 等价于 Dijkstra 式最短路搜索。

12.2 主要边权

使用题图中的权重，特别地 $12 \rightarrow T$ 的权重为 15。关键边包括：

$$S \rightarrow 1 : 8, \quad S \rightarrow 6 : 2, \quad S \rightarrow 7 : 3,$$

$$1 \rightarrow 2 : 11, \quad 1 \rightarrow 8 : 2,$$

$$6 \rightarrow 8 : 12, \quad 7 \rightarrow 9 : 8, \quad 8 \rightarrow 9 : 5, \quad 8 \rightarrow 11 : 18,$$

$$9 \rightarrow 11 : 10, \quad 11 \rightarrow 12 : 9, \quad 12 \rightarrow T : 15,$$

$$2 \rightarrow 3 : 7, \quad 2 \rightarrow 4 : 11, \quad 3 \rightarrow 5 : 6, \quad 4 \rightarrow 5 : 5, \quad 4 \rightarrow 10 : 4, \quad 10 \rightarrow 12 : 8, \quad 5 \rightarrow T : 15.$$

12.3 搜索过程

扩展顺序及代价如下：

扩展顺序	结点	$g(n)$	$h(n)$	$f(n)$
0	S	0	0	0
1	6	2	0	2
2	7	3	0	3
3	1	8	0	8
4	8	10	0	10
5	9	11	0	11
6	2	19	0	19
7	11	21	0	21
8	3	26	0	26
9	4	30	0	30
10	12	30	0	30
11	5	32	0	32
12	10	34	0	34
13	T	45	0	45

由前驱关系回溯得到最短路径：

$$S \rightarrow 7 \rightarrow 9 \rightarrow 11 \rightarrow 12 \rightarrow T.$$

总代价为

$$3 + 8 + 10 + 9 + 15 = 45.$$

13 单调栈求字典序最小推荐列表

13.1 问题转化

原始评分序列为

$$A = [9, 4, 7, 3, 5, 2, 6],$$

要求按原顺序保留 4 个商品，使结果字典序最小。等价于删除

$$7 - 4 = 3$$

个元素，使剩余子序列字典序最小。

13.2 贪心思想

为了让字典序最小，应尽量让靠前位置的元素更小。因此从左到右扫描序列时，若当前元素小于栈顶元素，并且仍有删除次数，则弹出栈顶元素。

13.3 单调栈过程

初始 $stack = []$, $drop = 3$ 。

处理元素	操作	栈状态	剩余删除次数
9	入栈	[9]	3
4	弹出 9, 4 入栈	[4]	2
7	入栈	[4, 7]	2
3	弹出 7、4, 3 入栈	[3]	0
5	入栈	[3, 5]	0
2	入栈	[3, 5, 2]	0
6	入栈	[3, 5, 2, 6]	0

最终结果为

$[3, 5, 2, 6]$ 。

13.4 伪代码

算法 6 单调栈求长度为 k 的字典序最小子序列

输入： 原序列 A , 目标长度 k

输出： 长度为 k 的字典序最小子序列

```

1:  $n \leftarrow |A|$ ,  $drop \leftarrow n - k$ 
2:  $Stack \leftarrow \emptyset$ 
3: for 每个元素  $x \in A$  do
4:   while  $Stack \neq \emptyset$  且  $drop > 0$  且  $top(Stack) > x$  do
5:     弹出  $top(Stack)$ 
6:      $drop \leftarrow drop - 1$ 
7:   end while
8:   将  $x$  压入  $Stack$ 
9: end for
10: while  $|Stack| > k$  do
11:   弹出栈顶元素
12: end while
13: return  $Stack$  的前  $k$  个元素

```

13.5 贪心与单调栈的配合

贪心解决的是选择原则：当前元素更小且还能删除时，应删除前面较大的元素，因为字典序优先比较靠前位置。

单调栈解决的是实现问题：它高效维护当前候选序列，并通过弹栈删除不优选择，同时保证元素相对顺序不变。

每个元素最多入栈一次、出栈一次，故时间复杂度为 $O(n)$ ，空间复杂度为 $O(n)$ 。